

DISPENSA DELLA LEZIONE

Condizioni, confronti e blocchi

if, confronti, input utente, numeri casuali e controllo del flusso.

CAPITOLO 1

if, else e catene di condizioni

NOTA

L'istruzione `if` esegue un blocco di codice solo se la condizione è vera. Con `else` si definisce cosa fare quando è falsa. Con `else if` si aggiungono controlli intermedi.

La condizione viene scritta tra parentesi tonde dopo `if`. JavaScript la valuta e ottiene un risultato: vero oppure falso. Se il risultato è vero, viene eseguito il blocco tra le parentesi graffe subito dopo; altrimenti viene saltato. In tutti i casi JavaScript fa sempre la stessa domanda implicita: **"questo valore è vero?"**.

`else` è il blocco alternativo: viene eseguito solo quando la condizione dell'`if` è falsa. I due blocchi si escludono sempre a vicenda: uno dei due viene eseguito, mai entrambi, mai nessuno.

`else if` permette di controllare un caso specifico prima di arrivare al fallback. I casi vengono valutati dall'alto verso il basso: il primo blocco la cui condizione è vera viene eseguito, tutti gli altri vengono saltati.

Esempio

```
let voto = 7;

if (voto >= 9) {
  console.log("Ottimo");
} else if (voto >= 7) {
  console.log("Buono");
} else if (voto >= 6) {
  console.log("Sufficiente");
} else {
  console.log("Insufficiente");
}
```

CAPITOLO 2

Math.random()

NOTA

`Math.random()` genera ogni volta un numero decimale casuale compreso tra `0` incluso e `1` escluso. Non restituisce mai esattamente `1`.

Un numero come `0.47` o `0.92` da solo è poco utile. Per ottenere un numero intero casuale in un intervallo preciso, servono due passaggi.

Il primo passaggio è moltiplicare il risultato per l'ampiezza dell'intervallo desiderato più uno. Per avere un numero da 0 a 100 compresi, si moltiplica per `101`: così il risultato va da `0` fino a quasi `101`, coprendo tutti e 101 i possibili interi. Il secondo passaggio è usare `Math.floor()`, che elimina la parte decimale arrotondando sempre per difetto. Il risultato finale è un numero intero casuale tra 0 e 100.

Esempio

```
let dado = Math.floor(Math.random() * 101);

if (dado > 50) {
  console.log("Numero sopra la metà: " + dado);
} else {
  console.log("Numero sotto la metà: " + dado);
}
```

CAPITOLO 3

La funzione `prompt()`

NOTA

`prompt()` apre una finestra di dialogo che chiede un dato all'utente. Mentre la finestra è aperta, l'esecuzione della pagina si ferma completamente e riprende solo dopo che l'utente ha risposto.

`prompt()` accetta due parametri. Il primo è il testo della domanda che appare nella finestra. Il secondo è facoltativo e imposta un valore già scritto nel campo di testo: serve per fare test più rapidi, senza dover riscrivere ogni volta la stessa risposta.

Il valore restituito è sempre una stringa, anche se l'utente ha scritto un numero. Se l'utente chiude la finestra senza rispondere, `prompt()` restituisce `null`, che è un valore falsy.

Esempio

```
let nome = prompt("Come ti chiami?", "Mario");

if (nome) {
  console.log("Ciao " + nome + ", benvenuto!");
} else {
  console.log("Nessun nome inserito.");
}
```

CAPITOLO 4

La funzione Number()

NOTA

`Number()` converte un valore in numero. È indispensabile quando si riceve un dato da `prompt()` e lo si vuole usare in un calcolo o in un confronto numerico.

Tutto quello che arriva da `prompt()` è una stringa. Scrivere `"18"` non è la stessa cosa di scrivere `18`: il primo è testo, il secondo è un numero. Se si confronta una stringa con un numero usando `===`, il risultato è sempre `false`, anche se i contenuti sembrano identici.

`Number()` trasforma la stringa nel corrispondente valore numerico. Se la conversione non è possibile, per esempio perché l'utente ha scritto una parola, il risultato è `NaN`, che sta per "Not a Number".

Esempio

```
let testoEta = prompt("Quanti anni hai?", "18");
let eta = Number(testoEta);

if (eta >= 18) {
  console.log("Sei maggiorenne");
} else {
  console.log("Sei minorenni");
}
```

CAPITOLO 5

if nested e blocchi annidati

NOTA

Un **if nested** è un **if** scritto dentro un altro **if**. Il blocco interno viene valutato solo se il blocco esterno è già risultato vero.

Gli **if** annidati servono quando la risposta a una domanda dipende da una condizione più generale già verificata. Prima si controlla il requisito principale, poi si aggiungono verifiche più specifiche all'interno.

È importante non abusare di questo pattern: più livelli di annidamento rendono il codice difficile da leggere. Due o tre livelli sono gestibili; oltre conviene ripensare la struttura.

Esempio

```
let haAccount = true;
let passwordCorretta = true;

if (haAccount) {
  if (passwordCorretta) {
    console.log("Accesso effettuato");
  } else {
    console.log("Password errata");
  }
} else {
  console.log("Account non trovato");
}
```

CAPITOLO 6

Operatori binari e confronto

NOTA

Gli operatori di confronto mettono a confronto due valori e restituiscono sempre **true** oppure **false**. Si chiamano binari perché lavorano su due operandi.

Il confronto stretto `===` è vero solo se i due valori sono identici sia nel contenuto sia nel tipo. Il confronto debole `==` invece tenta una conversione automatica del tipo prima di confrontare: `"5" == 5` è **true**, ma `"5" === 5` è **false**.

Nella pratica è sempre preferibile usare `===` e `!==`. Il confronto debole può produrre risultati inattesi e rendere il codice difficile da capire.

Esempio

```
let punteggio = 75;
let nome = "Anna";
let testoNumero = "75";

if (punteggio > 60) {
  console.log("Punteggio sufficiente");
}

if (punteggio >= 75) {
  console.log("Soglia esatta o superata");
}

if (nome === "Anna") {
  console.log("Nome corretto");
}

if (punteggio == testoNumero) {
  console.log("Confronto debole: vero");
}

if (punteggio !== Number(testoNumero)) {
  console.log("Questo non viene stampato");
} else {
  console.log("Confronto stretto tra numero e numero: vero");
}
```

CAPITOLO 7

Valori falsy e condizioni reali

NOTA

In JavaScript alcuni valori vengono considerati falsi anche senza essere `false`. Si chiamano **falsy**: `false`, `0`, `""`, `null`, `undefined` e `NaN`.

Quando si scrive `if (valore)`, JavaScript non controlla se `valore` è esattamente uguale a `true`. Controlla se il valore è *considerato vero*. Un valore falsy fa saltare il blocco, tutti gli altri lo eseguono.

Questo comportamento è molto comodo nella pratica: per esempio, si può controllare se un utente ha scritto qualcosa in un `prompt()` semplicemente con `if (risposta)`, senza dover confrontare esplicitamente con una stringa vuota.

Esempio

```
let risposta = "";
let quantita = 0;
let valore;

if (risposta) {
    console.log("L'utente ha scritto qualcosa");
} else {
    console.log("Stringa vuota: nessuna risposta");
}

if (quantita) {
    console.log("La quantità è valida");
} else {
    console.log("Zero è falsy: quantità non valida");
}

if (valore) {
    console.log("Il valore esiste");
} else {
    console.log("undefined è falsy: valore non impostato");
}
```

CAPITOLO 8

Operatori logici combinati

NOTA

&& (AND) è vero solo se entrambe le condizioni sono vere. **||** (OR) è vero se almeno una delle due condizioni è vera.

Gli operatori logici permettono di combinare più condizioni in un'unica espressione. Questo evita di scrivere **if** annidati per ogni singolo controllo e rende il codice più diretto e leggibile.

Quando si usano variabili booleane in una condizione, è buona abitudine scrivere il confronto per esteso, per esempio **piove === true** invece di **piove**. Questa forma rende immediatamente chiaro cosa si sta verificando.

Esempio

```
let piove = true;
let ombrelloAperto = false;
let haPatente = true;
let haMacchina = true;
let iscritto = false;
let haPagato = true;

if (piove === true && ombrelloAperto === false) {
  console.log("Stai prendendo la pioggia");
}

if (haPatente === true && haMacchina === true) {
  console.log("Puoi guidare");
}

if (iscritto === true || haPagato === true) {
  console.log("Accesso consentito");
}
```